

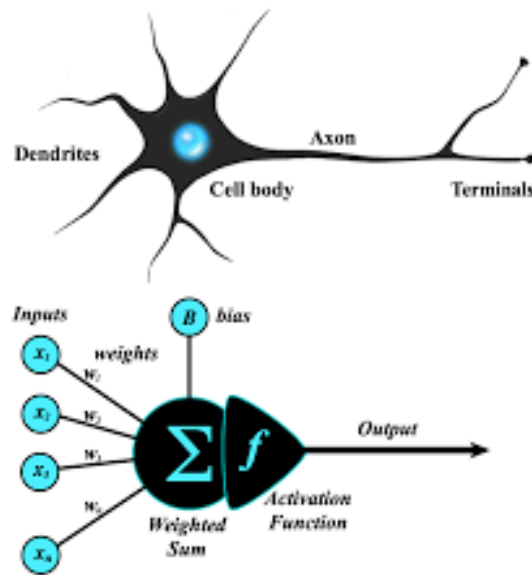


Figure 1: Single Neuron Compared with a Biological Neuron: Source. Islam, (2019)

0.1 Definition and Characteristics

Neural networks are designed using mathematical equations to mimic the biological neurons in the body system. Each neuron is designed to pick a signal, transmit to interpretation center, and send response signal, either to stimulate or suppress the original source of stimulus. Artificial Neural Networks (ANN) are computational models created using mathematical equations, that mimic biological neurons. These ANN are characterized by:

- Input Layer: Receives data.
- Hidden Layers: Process information.
- Output Layer: Produces results.

Watch the youtube video available in the link below and answer the following question. Youtube Discussion Questions

1. Which of the following statement does not old true?
 - Activation functions are threshold functions
 - Error is calculated at each layer of the Neural Network

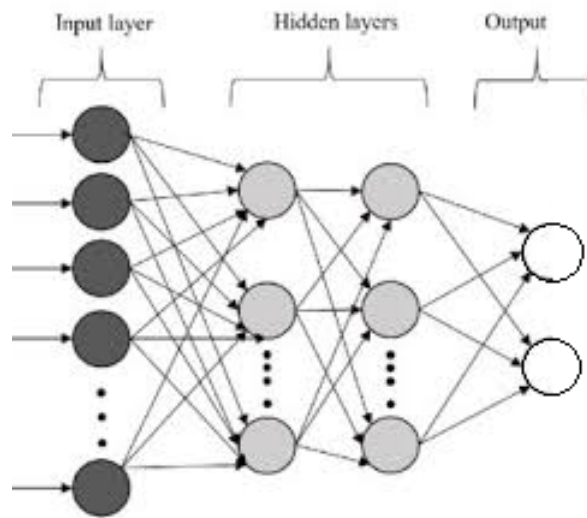


Figure 2: Neural Network Architecture showing Layers. Source: Islam, (2019)

- Both forward and back propagation take place during the training of Neural Network
- Most of the data processing is carried out in the hidden layer

2. List any three applications of neural networks in real life.

The specific form of NN model equations depends on the architecture of the neural network, such as whether it's a feedforward network, a convolutional network, a recurrent network, or a more complex architecture. The equations are used to represent the process of data propagation, learning, and decision-making within the neural network.

Example 1. 0.2 Practical Worked Example with Python

Bulding a simple XOR Problem ANN The XOR function is a classic classification problem solved using a neural network. A simple ANN with one hidden layer can learn to classify XOR inputs.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# Define model
```

```

model = Sequential([
    Dense(2, activation='relu', input_shape=(2,)),
    Dense(1, activation='sigmoid')
])

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Example input
import numpy as np
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 0])

# Train model
model.fit(X, y, epochs=100, verbose=0)

# Predict
print(model.predict(X))

```

Question: What is the primary function of an activation function in a neural network? Discussion Questions

1. How do artificial neural networks differ from traditional algorithms?
2. What are the advantages of using ANNs in simulation?

1 Neural Network Applications in Simulation

1.1 Definition and Characteristics

An artificial Neural Network ANN is a framework of interconnected artificial neurons forming layer. Each neuron receives an input signal, applies weights and passes the result through activation function. The output signal is adjusted using weights through backpropagation algorithms and gradient descent. Once the error is eliminated, the ANN will have correctly completed the task of image recognition, speech processing, music composition, financial forecasting, or medical diagnosis. Artificial Neural Networks (ANNs) are computational models that mimic biological neurons. They consist of:

- Input Layer: Receives data.
- Hidden Layers: Process information.
- Output Layer: Produces results.

Example 2. 1.2 Worked XOR Example using Python

*Consider a XOR problem of predicting Stock Prices. A Neural Network can be trained on historical stock data to predict future trends. **Python Stock Price Prediction Code***

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM

# Generate dummy data
X = np.random.rand(100, 10)
y = np.random.rand(100, 1)

# Define model
model = Sequential([
    Dense(10, activation='relu', input_shape=(10,)),
    Dense(1)
])

# Compile and train
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=50, verbose=0)

# Predict
print(model.predict(X[:5]))
```

Question: What is the primary function of an activation function in a neural network? Discussion Questions

1. What are the limitations of neural networks in simulation?
2. How can neural networks improve decision-making in engineering?

2 Training and Optimizing Neural Models

Training of neural network is the process which involves adjusting of weights and biases (parameters), so as to predict accurately using provided historical data sets. The data is subdivided into three, namely:

1. A set if input features X , which consist of data to be used to make prediction

2. A set of output target label, Y , which is a set of correct values of data targetted
3. A set of training, validation and test set explained below.
 - Training data is a set of data used to adjust the model parameters. This process is called training
 - This is a st of validation data, which refers to the process of moderating the fitting of the model to the data
 - Test set is the data used to check the performance of the model whether prediction is accurate

2.1 Optimization

Optimization problem involves the evaluation of an objective function, which could be maximization or minimization problem, subject to provided constraints. An optimization problem is formulated with the following components; Objective function, Constraints, and declaration of feasibility space or region. Neural networks are widely used in optimization problems due to their ability to learn complex patterns, approximate functions, and improve decision-making processes. Their applications span various fields, including engineering;(structural design, control systems, and smart grid energy mix), finance (stock price prediction, portfolio management, fraud detection), healthcare (diagnosis, cancer progression, drug discovery, patient management), and artificial intelligence (reinforcement learning, robotics, hyperparameter tunng, and neural architecture search).

2.2 Optimization Techniques

Neural networks require optimization for better performance. Some of optimization techniques used include:

- **Gradient Descent.** Gradient Descent is a process where the approach to the maximum or minimum poit of a function is based on the gradient (steepness or gentleness) of the slope. The step size towards the optimal point is based on the magnitude of the gradient function. This process is involved in the adjustment of weights to minimize error.
- **Regularization.** Regularization in neural network optimization technique used to prevent overfitting and improve the model's ability to generalize to unseen data. Overfitting occurs when a neural network

learns the training data too well, including noise, making it perform poorly on new data, or there are unforeseen small changes.

- **Hyperparameter Tuning.** These are the settings adjusted in the neural network model before training, to determine the rate of training, batch size of data used, and the level of accuracy of prediction. Hyperparameter tuning depends on the complexity of the model, accuracy of the expected results and the desired training time.

Example 3. *Using dropout layers and batch normalization improves model accuracy.* **Python Example: Optimizing a Neural Network**

```
from tensorflow.keras.layers import Dropout, BatchNormalization

# Define optimized model
model = Sequential([
    Dense(64, activation='relu', input_shape=(10,)),
    BatchNormalization(),
    Dropout(0.2),
    Dense(1)
])

# Compile and train
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=50, verbose=0)
```

Formulation of ANN depends on the complexity of the problem to be solved. Watch the Youtube video using the link below and attempt the discussion questions given hereunder. **Question:** What is the purpose of dropout layers in neural networks? Discussion Questions

1. How does regularization improve neural network performance?
2. What are common challenges in training neural networks?

Task: Develop a neural network-based simulation model to predict outcomes in a real-world scenario. The project will involve data collection, model training, optimization, and evaluation using Python. Here is an example of a NN Python Code used to predict Stock Market Prices for your guide.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```

from tensorflow.keras.layers import Dense, LSTM

# Generate dummy data
X = np.random.rand(1000, 10) # Features
y = np.random.rand(1000, 1)  # Target (Stock Price)

# Define neural network model
model = Sequential([
    Dense(64, activation='relu', input_shape=(10,)),
    Dense(32, activation='relu'),
    Dense(1) # Output layer
])

# Compile model
model.compile(optimizer='adam', loss='mse')

# Train model
model.fit(X, y, epochs=50, batch_size=32)

# Predict
predictions = model.predict(X[:5])
print("Predicted Stock Prices:", predictions)

```

3 END OF MODULE ASSESSMENT

1. What is the primary function of an activation function in a neural network?
 - A). To determine whether a neuron should be activated **Correct Answer**
 - B). To add random noise to inputs
 - C). To normalize input data
 - D). To generate weights for the network
2. Which Python library is most commonly used for building neural networks?
 - A). NumPy
 - B). Pandas
 - C). TensorFlow **Correct Answer**

- D). Matplotlib
3. What does backpropagation do in a neural network?
- A). Initializes weights
 - B). Updates weights based on errors **Correct Answer**
 - C). Eliminates neurons
 - D). Reduces computation speed
4. What is the role of a loss function in neural networks?
- A). Converts input into output
 - B). Measures the error between predicted and actual values **Correct Answer**
 - C). Adjusts the number of layers dynamically
 - D). Prevents overfitting
5. Which of the following techniques helps prevent overfitting in neural networks?
- A). Increasing the learning rate
 - B). Removing activation functions
 - C). Using only one layer
 - D). Using dropout layers **Correct Answer**

Answers to the project task

The following is a python code to answer the task in question 6 above.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST dataset
mnist = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values
x_train, x_test = x_train / 255.0, x_test / 255.0
```



```

# Build Neural Network Model
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)), # Input layer
    keras.layers.Dense(128, activation='relu'), # Hidden layer
    keras.layers.Dropout(0.2), # Dropout to prevent overfitting
    keras.layers.Dense(10, activation='softmax') # Output layer
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=5)

# Evaluate performance
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test Accuracy: {test_acc:.2f}")

# Predict a digit
prediction = model.predict(np.expand_dims(x_test[0], axis=0))
plt.imshow(x_test[0], cmap='gray')
plt.show()
print(f"Predicted Digit: {np.argmax(prediction)}")

```